

## 《数据库系统原理》实验报告（）

题目：miniOB 入门实验

|    |         |    |     |    |           |
|----|---------|----|-----|----|-----------|
| 学号 | 2452769 | 姓名 | 幸可函 | 日期 | 2026/4/21 |
|----|---------|----|-----|----|-----------|

实验环境：

1. 操作系统：Windows 11
2. 容器平台：Docker Desktop for Windows（版本：4.65.0）
3. 数据库镜像：oceanbase/miniob:latest
4. 数据库系统：miniOB

1.在 Docker 中建立 miniob 环境（参考第 0 章的教程 PPT 和第三节的内容）。

```
# cd build
# ./bin/observer -s miniob.sock -f ../etc/observer.ini &
#
Welcome to the OceanBase database implementation course.

Copyright (c) 2021 OceanBase and/or its affiliates.

Learn more about OceanBase at https://github.com/oceanbase/oceanbase
Learn more about MiniOB at https://github.com/oceanbase/miniob

Successfully load ../etc/observer.ini

# ./bin/obclient -s miniob.sock

Welcome to the OceanBase database implementation course.

Copyright (c) 2021 OceanBase and/or its affiliates.

Learn more about OceanBase at https://github.com/oceanbase/oceanbase
Learn more about MiniOB at https://github.com/oceanbase/miniob

miniob > 
```

2.创建一张表，包括学号，姓名，身高，体重。

| 表名：Student |    |          |
|------------|----|----------|
| 字段         | 说明 | 类型       |
| No         | 学号 | int      |
| Name       | 姓名 | char(10) |
| Height     | 身高 | float    |
| Weight     | 体重 | float    |

```
miniob > create table Student(No int, Name char(10), Height float, Weight float);
SUCCESS
```

3.向该表插入下列数据。

| No      | Name | Height | Weight |
|---------|------|--------|--------|
| 2353123 | 王海峰  | 180.5  | 75.5   |
| 2331902 | 刘伟   | 174.9  | 63.3   |
| 2353074 | 孙国程  | 169.0  | 82.5   |
| 2233444 | 严磊   | 178.3  | 52.8   |

```
miniob > insert into Student values (2353123, '王海峰', 180.5, 75.5);
SUCCESS
miniob > insert into Student values (2331902, '刘伟', 174.9, 63.3);
SUCCESS
miniob > insert into Student values (2353074, '孙国程', 169.0, 82.5);
SUCCESS
miniob > insert into Student values (2233444, '严磊', 178.3, 52.8);
SUCCESS
```

4.使用 select 语句展示学号，姓名，身高。

```
miniob > select No, Name, Height from Student;
No | Name | Height
2353123 | 王海峰 | 180.5
2331902 | 刘伟 | 174.9
2353074 | 孙国程 | 169
2233444 | 严磊 | 178.3
```

5.尝试修改指定行的体重如下表所示，能否成功？为什么？

| No      | Name | Height | Weight |
|---------|------|--------|--------|
| 2353123 | 王海峰  | 180.5  | 78.5   |
| 2331902 | 刘伟   | 174.9  | 60.0   |
| 2353074 | 孙国程  | 169.0  | 82.5   |
| 2233444 | 严磊   | 178.3  | 52.8   |

```
miniob > update Student set Weight = 78.5 where No = 2353123;
FAILURE
miniob > update Student set Weight = 60.0 where No = 2331902;
FAILURE
```

可能是当前 miniOB 实验环境中的执行功能尚不完整、版本实现受限

6.删除孙国程和严磊的记录。

```
miniob > delete from Student where Name = '孙国程';
SUCCESS
miniob > delete from Student where Name = '严磊';
SUCCESS
```

7.对 miniob 源码(<https://github.com/oceanbase/miniob>)进行阅读,主要选取一个功能(如 create table、insert、delete 等)进行分析理解, 做简要报告(不超过两页)。

本报告选取 miniOB 的 INSERT 功能进行分析。MiniOB 将一次 SQL 请求依次交给 Parser、Resolver、Optimizer、Executor 和 Storage Engine 等模块处理,因此插入语句并不是直接写文件,而是经历了解析、语义检查、计划生成、执行和存储落盘这几层步骤。

INSERT 的入口首先在解析阶段。parse(const char \*st, ParsedSqlResult \*sql\_result) 会调用 sql\_parse,把输入 SQL 转换为 ParsedSqlResult 中的语法树节点 (miniob/src/observer/sql/parser/parse.cpp at main · oceanbase/miniob)。之后在 Stmt::create\_stmt 中,根据 sql\_node.flag 分派到具体语句类型;当标志为 SCF\_INSERT 时,就会调用 InsertStmt::create(db, sql\_node.insertion, stmt),这说明 miniOB 在语法树之后还有一步语义对象化,把较原始的解析结果包装成更适合后续优化与执行的 Stmt 对象。(miniob/src/observer/sql/stmt/insert\_stmt.cpp at main · oceanbase/miniob)

InsertStmt::create 本身做了几项很关键的检查:先断 db、表名和插入值列表是否合法,然后通过 db->find\_table(table\_name) 查表是否存在;若表不存在,则返回 SCHEMA\_TABLE\_NOT\_EXIST。接着它取出表元数据 table\_meta,用表总字段数减去系统字段数的方式得到用户可见字段数,再与 INSERT 中提供的值个数比较;如果数量不一致,则返回 SCHEMA\_FIELD\_MISSING。也就是说,miniOB 在真正执行插入前,已经先完成了最基本的模式检查,避免不合法数据进入执行层。

通过语义检查后,系统进入计划生成阶段。逻辑计划生成器 LogicalPlanGenerator::create\_plan(InsertStmt \*...) 会从 InsertStmt 中取出表对象和待插入的值,构造一个 InsertLogicalOperator;随后物理计划生成器 PhysicalPlanGenerator::create\_plan(InsertLogicalOperator &...) 再把它变成 InsertPhysicalOperator。(miniob/src/observer/sql/optimizer/logical\_plan\_generator.cpp at main · oceanbase/miniob)

真正执行插入时,核心在 InsertPhysicalOperator::open(Trx \*trx) (miniob/src/observer/storage/trx/trx.h at main · oceanbase/miniob)。这里先调用 table->make\_record(...) 把一组 Value 组装成底层 Record;make\_record 会依据表元数据分配记录缓冲区,逐字段检查类型,如果类型不一致则尝试 Value::cast\_to 做隐式转换,随后按字段偏移写入记录内存,最后由 record.set\_data\_owner 接管这段数据。这个过程说明 miniOB 的行记录本质上是一块按 schema 布局好的二进制内存,而不是简单的对象数组。

记录组装完成后,插入请求并不直接写页,而是交给事务层 trx->insert\_record(table\_, record)。在接口设计上,Trx 抽象类规定了 insert\_record 行为;若使用 VacuumTrx,插入会直接转发给 table->insert\_record(record),几乎不做事务控制。若使用 MvccTrx,则会先取得事务字段,把 begin\_xid 设为负的当前事务 id,把 end\_xid 设为最大事务 id,再执行表插入、追加插入日志,并把这次操作记录到 operations\_ 中;在提交时,再把先前写入的负事务 id 改成真正的提交 id。

继续向下，表插入最终会落到记录管理模块。RecordFileHandler::insert\_record 会先从 free\_pages\_ 中寻找未满页；如果没有可用页，就分配新页并初始化，然后把页号加入空闲页集合，最后调用页处理器执行真正的插入。对于行存页面，RowRecordPageHandler::insert\_record 会先检查页面是否已满，再寻找空闲槽位，设置该位、增加记录数、写入日志、把数据 memcpy 到对应槽位，并设置返回的 RID(page\_num, slot\_num)，同时把 frame 标记为 dirty。由此可以看出，miniOB 的插入在最底层实际上就是“找空槽—拷贝字节—更新元信息—记日志”的过程。

(miniob/src/observer/storage/record/record\_manager.cpp at main · oceanbase/miniob)

总的来看，miniOB 的 INSERT 功能虽然代码量不大，但已经完整体现了：生成语法树，Stmt 层完成语义检查，优化器生成逻辑/物理算子，执行器将值组装成记录，事务层决定可见性与日志行为，最后记录管理模块把数据写入具体页面的一条链步骤。我认为这个实现最值得学习的地方，在于它把数据库系统中几个关键抽象行为——语句对象、执行计划、事务、记录页——用一条清晰的调用链串联了起来，非常适合作为理解数据库内核的入门案例。

出现的问题：

1. miniOB 不支持插入数据时使用一行多元组的写法：

```
miniob > insert into Student values (2353123, '王海峰', 180.5, 75.5),(2331902, '刘伟', 174.9, 63.3),(2353074, '孙国程', 169.0, 82.5),(2233444, '严磊', 178.3, 52.8);
SUCCESS
miniob > select No, Name, Height from student;
FAILURE
miniob > select No, Name, Height from Student;
No | Name | Height
2353123 | 王海峰 | 180.5
```

可以看到这里只成功插入了一条数据

解决方案：

1. 将原插入语句拆分成多行：

```
miniob > insert into Student values (2353123, '王海峰', 180.5, 75.5);
SUCCESS
miniob > insert into Student values (2331902, '刘伟', 174.9, 63.3);
SUCCESS
miniob > insert into Student values (2353074, '孙国程', 169.0, 82.5);
SUCCESS
miniob > insert into Student values (2233444, '严磊', 178.3, 52.8);
SUCCESS
```